Universidad Nacional Autónoma de México

Ingeniería en computación

COMPILERS

LEXER

TEAM 06 MEMBERS:

423054589 - Arroyo Onofre Leonardo

319061345 - Camacho Ignacio Violeta

320047822 - Castañeda Luna Valeria

423020441 - Martínez Pérez Diego Alexander

320263635 - Ruiz Ramírez Santiago

320284858 - Saucedo Serralde Alejandra Melissa

GROUP: #5
SEMESTER:
2026-II

México,CDMX. March 2026

Índice

1. Introduction **2**

2. Theoretical framework **3**

3. Development **4**
3.1. Test 1 5
3.2. Test 2 5
3.3. Test 3 5
3.4. Test 4 6
3.5. Test 5 6
3.6. CFG with Left Factoring 6

4. Results **7**
4.1. Test 1 7
4.2. Test 2 7
4.3. Test 3 8
4.4. Test 4 8
4.5. Test 5 8
4.6. CFG Step-by-step Derivation 9

5. Conclusions **9**

6. References **10**

1. Introduction

The challenge of this project is to implement a lexical analyzer capable of recognizing at least forty distinct tokens, including keywords, identifiers, operators, constants, strings, and punctuation marks, as well as to define a context-free grammar with left factoring for representative examples.

Lexical analysis is the foundational stage of a compiler and its function is to transform the source code into a sequence of tokens that represent the minimal units of the language. Providing a solution to this problem, that is, constructing a lexical analyzer, is necessary because without it the compiler would not be able to distinguish between keywords, identifiers, operators, constants, strings, or punctuation marks. The lexer organizes the code into recognizable components, detects early errors, and prepares the input for the subsequent stages of syntactic and semantic analysis. In this way, the compiler can correctly interpret the program, and the theoretical concepts of formal languages are translated into a practical and functional implementation.

By implementing the lexical analyzer, it is expected to obtain a functional lexer capable of recognizing forty distinct tokens, including keywords, identifiers, operators, constants, strings, and punctuation marks.

- **Problem statement.** Before a program can be analyzed or executed by a compiler the source code must be transformed from a sequence of characters into meaningful elements, however, the raw text of a program does not initially contain an explicit structure that allows the compiler to interpret it.

  The problem addressed in this project is the design and implementation of a lexical analyzer capable of reading a source code input and identifying the basic tokens that compose it, these tokens include keywords, identifiers, operators, constants, literals, and punctuation symbols, additionally, the analyzer must count the total number of tokens detected in the input.

  By recognizing and classifying these elements the lexical analyzer provides the first structured representation of the program, which is necessary for further stages of the compilation process.

- **Motivation.** Lexical analysis is a fundamental stage in the compilation process because it allows the source code to be transformed into meaningful elements that can be processed by later stages of the compiler, without this step the compiler would not be able to distinguish between the different components of a program.

  The motivation for this project is to apply the theoretical concepts studied in class, particularly those related to lexical analysis and formal language processing in order to develop a functional lexical analyzer, by implementing a lexer it becomes possible to understand how source code is analyzed and structured during the early stages of compilation, as well as the importance of token recognition in the correct interpretation of a program.

- **Objective.** The objective of this project is to design and implement a lexical analyzer capable of reading a source code input and identifying the tokens

that compose it, the system should recognize different token categories such as keywords, identifiers, operators, constants, literals, and punctuation symbols.

Additionally the lexer must generate an output that displays the sequence of detected token types and the total number of tokens found in the input, by achieving this the project demonstrates the practical application of lexical analysis concepts and provides the first structured representation of the source code for further stages of compilation.

2. Theoretical framework

A compiler is a software system that is responsible for translating programs written in a high-level programming language into another language, typically machine code or an intermediate representation that can be executed by a computer, this translation is carried out through several stages among which lexical, syntactic, and semantic analysis are the most important, each stage processes the source program progressively until a representation is suitable for execution is obtained [1].

The first stage of this process is lexical analysis, performed by a component known as a lexical analyzer or lexer, this reads the source code as a stream of characters and groups them into meaningful units called tokens, these represent the smallest elements of a programming language, such as keywords, identifiers, operators, constants, literals, and punctuation symbols, during this stage the lexer may also remove elements that are not relevant for later stages such as whitespace and comments.[2]

A lexeme is the sequence of characters in the source code that matches the pattern of a token, for example, a reserved word like int, an identifier such as x, or a constant such as 10 are lexemes that correspond to different token categories, token recognition is commonly implemented using pattern matching techniques, such as regular expressions or finite automat, which allow the lexer to identify valid sequences of characters in the input program [3],[4].

Once the tokens are generated they are passed to the syntactic analysis phase, this stage verifies that the sequence of tokens follows the rules of the programming language, these rules are usually defined using Context-Free Grammars (CFGs), which describe the structure of valid sentences in the language using production rules [5].

A context-free grammar is typically defined as a tuple G=(V,T,P,S) where V is the set of non-terminal symbols, T is the set of terminal symbols, P is the set of production rules, and S is the start symbol, in order to simplify parsing and avoid ambiguities, grammar transformation techniques such as left factoring may be applied, left factoring restructures grammar rules that share common prefixes, making them more suitable for predictive parsing methods [5].

These theoretical concepts reviewed in class provided the foundation for the development of the lexical analyzer implemented in this project, which reads an input file, identifies lexemes, and classifies them into token categories for further processing in the compilation pipeline.

3. Development

First, the analyzer defines the lexical categories of the language. These categories include reserved words (keywords), identifiers, numeric constants, string literals, operators, punctuation symbols, comments, and whitespace. Each category represents a type of token that can appear in the source code.

Next, the analyzer prepares a collection of pattern definitions used to recognize each token type. These patterns are expressed as regular expressions, which allow the program to detect specific character sequences that correspond to valid tokens.

After defining the token patterns, the program reads the entire source file into memory. The analyzer then scans the input text sequentially, attempting to match one of the predefined patterns at the current position in the string. Whenever a pattern matches, the corresponding fragment of text is identified as a token and the scanning position moves forward.

This process continues until the end of the input is reached.

Once a fragment of text is recognized, the program determines the category of the token. Each recognized fragment is mapped into a higher-level classification used by the compiler.

For example:

- Words that match reserved language keywords are classified as keywords.
- Character sequences that follow identifier naming rules but are not reserved words are classified as identifiers.
- Numeric values such as integers, floating-point numbers, or hexadecimal values are classified as constants.
- Text enclosed in quotation marks or single quotes is classified as a literal.
- Mathematical or logical symbols are categorized as operators.
- Symbols that structure the program, such as parentheses or commas, are treated as punctuation.

This classification step converts raw text fragments into standardized token types that the rest of the compiler can easily process.

An important responsibility of lexical analysis is handling elements that do not contribute to program semantics, such as whitespace and comments.

Whitespace characters (spaces, tabs, and line breaks) are used only to improve readability in source code. Similarly, comments provide explanations for human readers but are not relevant to the program's execution. For this reason, the analyzer detects these elements and simply ignores them instead of generating tokens.

By filtering out these non-significant components, the analyzer ensures that the resulting token stream contains only meaningful language constructs.

The analyzer operates using a sequential scanning strategy. At each step, it attempts to match the longest possible token that begins at the current position in the

input string. Once a token is recognized, the scanning pointer advances by the length of that token.

This approach ensures that the analyzer processes the input deterministically and avoids overlapping matches. The program repeats this procedure until every character in the source file has been examined.

During the process, the analyzer records the type of each token found. At the end of the scan, the program outputs the sequence of token categories along with the total number of tokens identified.

To verify the correct behavior of the lexical analyzer, several tests were conducted using small fragments of C code as input. Each test evaluates whether the analyzer correctly identifies and classifies lexical elements such as keywords, identifiers, constants, operators, literals, and punctuation symbols. The tests also confirm that whitespace and comments are ignored during tokenization.

3.1. Test 1

Input

```
int a = 5;
```

Output

```
keyword identifier operator constant punctuation
Total of tokens: 5
```

3.2. Test 2

Input

```
float value = 3.14;
```

Output

```
keyword identifier operator constant punctuation
Total of tokens: 5
```

3.3. Test 3

Input

```
printf("Hello World");
```

Output

```
keyword punctuation literal punctuation punctuation
Total of tokens: 5
```

3.4. Test 4

Input

```
if (x >= 10) x++;
```

Output

```
keyword punctuation identifier operator constant punctuation
identifier operator punctuation
Total of tokens: 9
```

3.5. Test 5

Input

```
// comment
int x = 0;
```

Output

```
keyword identifier operator constant punctuation
Total of tokens: 5
```

3.6. CFG with Left Factoring

The following CFG represents the reading example int a = 5; used in test 1, the terminal symbols are in lowercase, while the non-terminal symbols are in capital letters.

$$
\begin{aligned}
S &\rightarrow \text{DECLARATION} \\
\text{DECLARATION} &\rightarrow \text{TYPE identifier DECLARATION}' \\
\text{DECLARATION}' &\rightarrow = \ \text{EXPRESSION ; } \mid \text{ ;} \\
\text{EXPRESSION} &\rightarrow \text{constant} \mid \text{identifier} \\
\text{TYPE} &\rightarrow \texttt{int} \mid \texttt{float} \mid \texttt{char} \mid \texttt{double}
\end{aligned}
$$

The reason we decide to make it this way is because if we use for example.

$$\text{DECLARATION} \rightarrow \text{TYPE identifier } = \text{ EXPRESSION ; } \mid \text{TYPE identifier}$$

This would only bring us an ambiguous grammar due to a shared common prefix, a parser would face a non-deterministic choice upon reading the TYPE and identifier tokens, it would not know which of the two production rules to choose without looking ahead By applying left factoring to the DECLARATION' rule we isolated it and DECLARATION' was created, in order to handle the divergent suffixes. With his transformations the parser can determine exactly which production to apply.

4. Results

The following figures present the execution results of the lexical analyzer. Each example shows a screenshot of the input provided to the analyzer and the corresponding output produced by the program. These captures illustrate how the lexer classifies tokens according to the defined lexical categories while ignoring whitespace and comments.

4.1. Test 1

The following screenshots show the input provided to the lexical analyzer and the resulting token classification generated by the program.

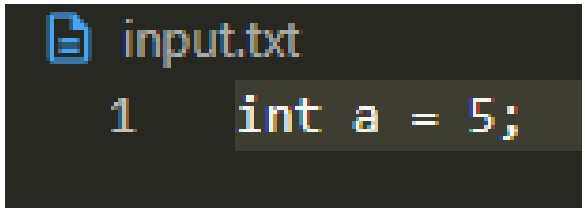

Figura 1: Input from Test 1

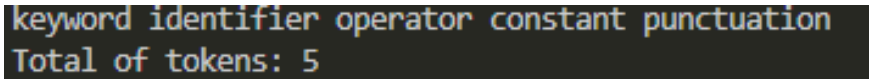

Figura 2: Output from Test 1

4.2. Test 2

The next images show the execution of the analyzer with the second input and the tokens identified in the output.

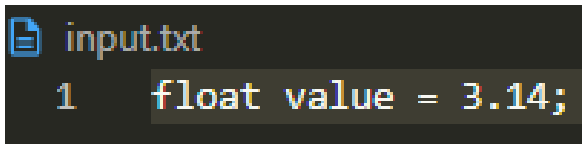

Figura 3: Input from Test 2

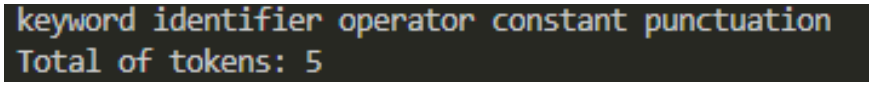

Figura 4: Output from Test 2

4.3. Test 3

These screenshots illustrate how the lexical analyzer processes literals and function calls within the source code.

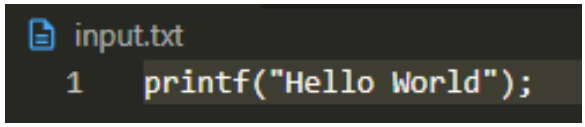

Figura 5: Input from Test 3

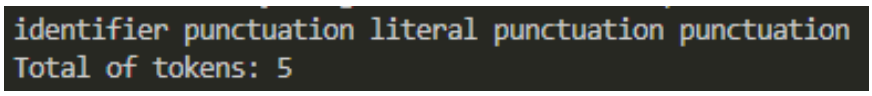

Figura 6: Output from Test 3

4.4. Test 4

The following figures show the analysis of conditional expressions and increment operators.

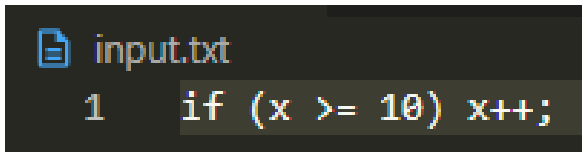

Figura 7: Input from Test 4

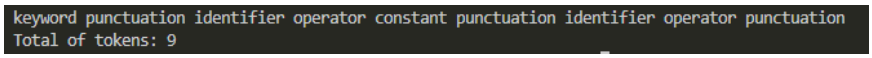

Figura 8: Output from Test 4

4.5. Test 5

Finally, the following screenshots demonstrate how the analyzer ignores comments and correctly processes the remaining tokens in the input.

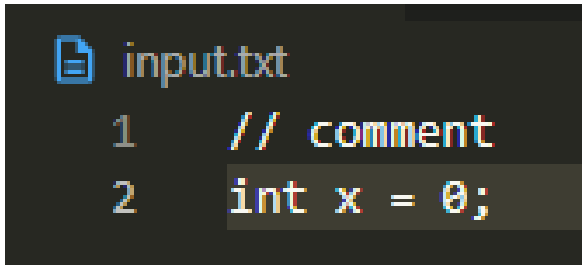

Figura 9: Input from Test 5

```
keyword identifier operator constant punctuation
Total of tokens: 5
```

Figura 10: Output from Test 5

4.6. CFG Step-by-step Derivation

The following table shows how the grammar derives the input int a = 5; step by step, applying the leftmost derivation:

Step	Sentential Form	Rule Applied
1	S	Start symbol
2	DECLARATION	$S \to$ DECLARATION
3	TYPE identifier DECLARATION$'$	DECLARATION $\to$ TYPE identifier DECLARATION$'$
4	`int` identifier DECLARATION$'$	TYPE $\to$ `int`
5	`int identifier` = EXPRESSION ;	DECLARATION$'$ $\to$ = EXPRESSION ;
6	`int identifier` = constant ;	EXPRESSION $\to$ constant
7	`int a` = constant ;	identifier $\to$ `a`
8	`int a = 5;`	constant $\to$ `5`

Tabla 1: Leftmost derivation of `int a = 5;`

5. Conclusions

The problem addressed in this project, transforming code into a structured sequence of tokens, required bridging the gap between the theoretical concepts studied in class and a functional software implementation, as this project demonstrates, this was made through design decisions like, how patterns are ordered, how conflicts between overlapping tokens are resolved, and how the boundary between scanning and classification is maintained consistently throughout the input.

The theoretical principles of regular expressions and pattern matching, initially seen and studied as abstract concepts, directly resulted in the creation of a reliable and predictable lexer. Any deviation from the theory leads to incorrect or inconsistent token recognition. The completeness of token coverage is not only listing categories, but defining their boundaries precisely enough that no input sequence falls outside the expected classification.

The CFG checks if the tokens form a valid structure and left factoring makes explicit that validity alone is insufficient if the grammar cannot be parsed efficiently. All of these components seen previously together reflect what the problem statement anticipated, that lexical analysis is the first structured representation of a program, and that this must be correct, since this is what makes every subsequent stage of a compiler possible.

6. References

[1] IBM, "What is a compiler?," *IBM Think*, [Online].
Available: https://www.ibm.com/mx-es/think/topics/compiler. [Accessed: Mar. 2026].

[2] GeeksforGeeks, "Introduction of Lexical Analysis," *GeeksforGeeks*, [Online].
Available: https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/. [Accessed: Mar. 2026].

[3] GeeksforGeeks, "Token, Patterns, and Lexemes," *GeeksforGeeks*, [Online].
Available: https://www.geeksforgeeks.org/compiler-design/token-patterns-and-lexems/. [Accessed: Mar. 2026].

[4] TutorialsPoint, "Compiler Design - Lexical Analysis," *TutorialsPoint*, [Online].
Available: https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm. [Accessed: Mar. 2026].

[5] GeeksforGeeks, "What is Context-Free Grammar?," *GeeksforGeeks*, [Online].
Available: https://www.geeksforgeeks.org/theory-of-computation/what-is-context-free-grammar/. [Accessed: Mar. 2026].